

Smart Civic Grievance Reporting and Tracking System Using AI

A Minor Project Report
Submitted in partial fulfillment of requirement of the
Degree of
**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**
BY
MAYANK PAWAR
EN23cs301606

Under the Guidance of
Dr.SHEETAL BAWANE
Prof . GARIMA TUKRA



Department of Computer Science & Engineering
Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331

APRIL 2026

Report Approval

The project work“Smart Civic Grievance Reporting and Tracking System Using AI” is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name: Dr.SHEETAL BAWANE

Designation

Affiliation

Declaration

I/We hereby declare that the project entitled “Smart Civic Grievance Reporting and Tracking System Using AI” submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications in ‘COMPUTER SCIENCE & ENGINEERING’ completed under the supervision of <**Dr. Sheetal Bawane**, COMPUTER SCIENCE & Faculty of Engineering, Medi-Caps University Indore is an authentic work. Further, I/we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

MAYANK PAWAR 22/04/2026

Certificate

I/We, Dr .Sheetal Bawane certify that the project entitled “Smart Civic Grievance Reporting and Tracking System Using AI” submitted in partial fulfillment for the award of the degree of Bachelor of Technology/Master of Computer Applications by Mayank pawar is the record carried out by him/them under my/our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr .Sheetal Bawane
COMPUTER SCIENCE & ENGINEERING
Medi-Caps University, Indore

Prof. (Dr.) Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
Medi-Caps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medi-Caps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Dr. Ratnesh Litoriya**, Dean, Faculty of Engineering, Medi-Caps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Prof. (Dr.) Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

MAYANK PAWAR

B.Tech. III Year (A)
Department of Computer Science & Engineering
Faculty of Engineering
Medi-Caps University, Indore

Abstract

CivicConnect is a full-stack web application designed to modernise the relationship between citizens and their local municipal bodies. The platform replaces fragmented, manual complaint processes — like phone calls, paper forms, or social media posts — with a structured, transparent, and digitally audited workflow.

Built using Python Flask with a MongoDB NoSQL backend, CivicConnect follows a clean Blueprint-based MVC architecture. Citizens can register, submit geo-tagged complaints with photo evidence, and track resolution progress in real time. Department staff manage complaints assigned to their domain and update statuses with work evidence. The administrator has complete system oversight, analytics, and data export capabilities.

The system integrates Ollama — a locally hosted AI engine — for duplicate detection, category suggestion, and a citizen chatbot. Security is addressed through CSRF double-submit cookie protection, MongoDB-backed IP rate limiting, TOTP two-factor authentication for admin and department accounts, bcrypt password hashing, OTP email verification, and session fixation prevention. Email notifications are delivered via Brevo (Sendinblue), PDFs are generated using ReportLab, and real-time updates are pushed through Flask-SocketIO with Redis as the message broker.

This report covers the complete design, architecture, implementation, and testing of CivicConnect — including all UML diagrams, database schema, API reference, security architecture, and a working system demonstration.

Table of Contents

Chapter 1 Introduction

- 1.1 Introduction
- 1.2 Literature Review
- 1.3 Objectives
- 1.4 Significance
- 1.5 Research Design
- 1.6 Source of Data
- 1.7 Chapter Scheme

Chapter 2 Requirements Specification

- 2.1 User Characteristics
- 2.2 Functional Requirements
- 2.3 Dependencies
- 2.4 Performance Requirements
- 2.5 Hardware Requirements
- 2.6 Constraints and Assumptions

Chapter 3 Design

- 3.1 System Architecture
- 3.2 Use Case Diagram
- 3.3 Activity Diagram
- 3.4 Sequence Diagram
- 3.5 Class Diagram
- 3.6 ER Diagram
- 3.7 State Transition Diagram
- 3.8 Database Design
- 3.9 API Reference
- 3.10 Security Architecture
- 3.11 Interface Design

Chapter 4 Implementation, Testing, and Maintenance

- 4.1 Technologies Used
- 4.2 Testing Techniques and Test Plans
- 4.3 Installation Instructions
- 4.4 End User Instructions

Chapter 5 Results and Discussions

- 5.1 Modules Overview
- 5.2 User Interface Walkthrough
- 5.3 Backend and Database

Chapter 6 Summary and Conclusions

Chapter 7 Future Scope

- Appendix A — Glossary
- Appendix B — TBD List
- Bibliography

Chapter 1: Introduction

1.1 Introduction

Municipal governance in most Indian cities still heavily relies on informal, fragmented complaint-handling processes. Citizens resort to phone helplines, paper submissions, or social media posts when they face civic issues like potholes, broken streetlights, or water supply failures. These approaches lack accountability — there is no structured tracking, no guaranteed feedback loop, and no visibility into what happens to a complaint after it is filed.

CivicConnect was conceived to solve this problem. It is a civic grievance management web platform that provides a clean, digital, and fully audited channel for citizens to report civic issues and track their resolution in real time. The platform auto-routes complaints to the right municipal department, enforces service-level deadlines, and notifies citizens at every step. The administrator has a bird's-eye view of the entire system — including analytics, exports, and departmental performance metrics.

At its core, CivicConnect is built on Python Flask with a MongoDB NoSQL database, following a Blueprint-based Model-View-Controller (MVC) architecture. It integrates locally hosted AI through Ollama for category suggestion and duplicate complaint detection, and uses Brevo (Sendinblue) for transactional email delivery. Security is a first-class concern — the system implements CSRF protection, rate limiting, two-factor authentication, and a comprehensive activity audit log.

1.2 Literature Review

Several existing civic complaint systems were studied before designing CivicConnect:

- FixMyStreet (mySociety, 2012) — a well-known UK platform for reporting street problems. It demonstrated the power of map-based reporting and category routing but lacked AI-driven features and real-time updates.
- SeeClickFix (2023) — a US-based civic engagement platform with strong geolocation features but no local AI integration or SLA enforcement mechanism.
- Jan Samasya (Government of India, pgportal.gov.in) — India's public grievance portal. While government-endorsed, it lacks modern UX, real-time status tracking, and smart AI assistance.
- Sharma et al. (IEEE ICSCT 2022) — demonstrated the feasibility of using sentence transformers for auto-classification of civic complaints, directly informing the NLP design of the AI service module.
- Gupta & Mehta (J. E-Governance Res., 2023) — explored the use of local LLMs (Ollama) for budget-constrained municipal AI, validating the choice of llama3.2 as the AI backend.
- Singh et al. (IJIS, 2023) — highlighted security vulnerabilities in Indian e-governance portals, directly shaping the defence-in-depth security architecture of CivicConnect.

The gap across all reviewed systems is consistent: no existing Indian civic platform combines structured digital workflows, locally hosted AI, real-time WebSocket updates, SLA enforcement, and a strong security architecture. CivicConnect is designed to fill this gap.

1.3 Objectives

The primary objectives of CivicConnect are:

- Provide citizens with a simple, accessible interface to report civic issues with geolocation, category tagging, and photo evidence.

- Automatically route complaints to the responsible municipal department based on complaint category.
- Give department staff a dedicated portal to update complaint statuses, attach work evidence, and add progress comments.
- Notify citizens by email at every status transition so they are always informed.
- Equip administrators with full complaint oversight, system-wide analytics, departmental management, and bulk data export capabilities.
- Integrate a free, locally hosted AI assistant (Ollama/llama3.2) for category suggestion, duplicate detection, complaint summarisation, and a citizen chatbot.
- Enforce strong security at every layer including CSRF protection, rate limiting, TOTP two-factor authentication, input sanitisation, and hashed credentials.
- Track SLA (Service Level Agreement) deadlines and automatically escalate overdue complaints.

1.4 Significance

CivicConnect addresses a genuine gap in Indian urban governance. By replacing ad-hoc processes with a structured digital platform, it brings several significant improvements:

- **Accountability:** every complaint is tracked from submission to resolution, with a full activity audit log.
- **Transparency:** citizens always know the status of their complaint and the expected resolution deadline.
- **Efficiency:** automatic routing eliminates manual triage; SLA tracking creates urgency for departments.
- **Data-driven governance:** the admin analytics dashboard gives municipal managers a data-backed view of civic health and departmental performance.
- **Privacy-first AI:** because Ollama runs locally, citizen complaint data never leaves the municipal server — a significant advantage over cloud AI APIs.
- **Scalability:** the S3-backed image storage and Redis-based WebSocket brokering make the system ready for horizontal scaling.

1.5 Research Design

CivicConnect is developed using an Object-Oriented design methodology with an Agile-inspired iterative process. Development was structured in feature sprints:

- **Sprint 1: Authentication** — registration with OTP, login, password reset
- **Sprint 2: Complaints** — submission, routing, editing, deletion, PDF generation
- **Sprint 3: Admin Panel** — complaint management, analytics, exports, department management
- **Sprint 4: Department Portal** — scoped complaint management, status updates, department analytics
- **Sprint 5: AI Integration** — Ollama chatbot, duplicate detection, category classification
- **Sprint 6: Security Hardening** — TOTP 2FA, account lockout, session fixation prevention, security headers
- **Sprint 7: SLA Tracking and Real-Time Updates** — APScheduler, Flask-SocketIO, Redis
- **Sprint 8: Testing** — pytest suite covering auth, complaints, security, and AI service functions

1.6 Source of Data

The system's data is generated entirely through user interactions — citizen registrations, complaint submissions, departmental status updates, and administrator actions. All data is stored in MongoDB collections within the civicdb database. There is no reliance on external civic data sources. Complaint images are stored either locally (development) or on AWS S3 (production). Email delivery is handled via the Brevo transactional email API.

1.7 Chapter Scheme

Chapter 2 covers the full requirements specification including user characteristics, functional requirements, dependencies, performance requirements, hardware requirements, and constraints.

Chapter 3 covers the complete design model including system architecture, all UML diagrams (Use Case, Activity, Sequence, Class, ER, State Transition), database design, API reference, security architecture, and interface design.

Chapter 4 covers implementation details, technologies used, the testing strategy and test plans, installation instructions, and end user guidance.

Chapter 5 presents the results and discussion — a walkthrough of all modules, UI screenshots, and database representations.

Chapter 6 provides the summary and conclusion. Chapter 7 outlines the future scope.

Chapter 2: Requirements Specification

2.1 User Characteristics

CivicConnect serves three distinct user groups, each with a dedicated portal and role-specific capabilities:

Role	Description	Key Capabilities
Citizen	Registered residents of the municipality	Submit complaints, track status, download PDF receipts, use AI assistant, manage profile and avatar
Department Staff	Municipal department employees (7 departments)	View assigned complaints, update status with work comments and evidence photos, access department analytics and SLA tracking
Administrator	Single system administrator (admin@civic.gov)	Full system oversight, complaint management, analytics, CSV/Excel exports, department account management, SLA configuration

Citizens are expected to have basic digital literacy — ability to use a web browser, fill forms, and interact with a map interface. Department staff and administrators are expected to be comfortable with a dashboard-style web application. All portals are designed to be accessible from both desktop and mobile browsers.

2.2 Functional Requirements

FR-01: Citizen Registration and Authentication

- Citizens register with name, email, and password. A 6-digit OTP is emailed via Brevo and must be verified within 10 minutes to activate the account.
- Login is available through a citizen portal (separate from department/admin portals). Department credentials are rejected on the citizen portal.
- Forgot password flow generates a 32-byte cryptographic token with 30-minute TTL. Response is always generic to prevent email enumeration.

FR-02: Complaint Submission

- Citizens fill a complaint form with title, description, category (from 8 predefined options), location text, and GPS coordinates from a Leaflet.js map (mandatory).
- Up to 5 photos can be uploaded as evidence; stored to AWS S3 in production or local disk in development.
- AI (sentence-transformers) checks for duplicate complaints (cosine similarity > 0.85) and displays a warning dialog. Citizens can still proceed if their complaint is genuinely distinct.
- Ollama llama3.2 suggests a category based on natural language text. The keyword fallback remains if Ollama is unavailable.
- On submission, the complaint is auto-routed to a department via the CATEGORY_DEPT_MAP dictionary. An SLA deadline is computed from sla_configs.
- A unique human-readable complaint ID is generated (format: CMP-YYYYMMDD-XXXXXX).

FR-03: Complaint Tracking and Management

- Citizens can view all their complaints with status indicators, SLA deadline display, and real-time WebSocket updates.

- Citizens can edit or delete their own complaints only if the complaint has not yet been assigned to a department.
- Citizens can download a styled PDF receipt with a scannable QR code linking to the live complaint tracking URL.

FR-04: Department Portal

- Department staff see only complaints assigned to their department — isolation enforced at the database query level.
- Status updates (Pending → In Progress → Resolved / Rejected) can be accompanied by a work comment (max 1000 chars) and up to 5 evidence photos.
- Every update is logged in the `complaint_activity` collection, triggers an email to the citizen, and emits a real-time WebSocket event.
- Department dashboard shows SLA compliance rate, escalated complaint count, and urgency colour-coded complaint cards.

FR-05: Admin Panel

- Administrators can view, filter (by status, priority, category, department, date range, overdue flag), and update any complaint system-wide.
- Analytics dashboard shows complaints by category, status distribution, departmental performance, and resolution-time statistics per department.
- Bulk export to CSV (13 columns) or Excel (13 columns + Summary sheet) with optional date-range filtering.
- Department account management — create, view, and impersonate department portals.
- SLA configuration panel to define max resolution hours per department-category-priority combination.
- Weekly digest emails sent automatically (APScheduler, daily at 08:00) to each department head and the administrator.

FR-06: AI Integration

- Citizen chatbot powered by Ollama (llama3.2) — answers civic questions and helps draft better complaints.
- Duplicate detection using sentence-transformers (all-MiniLM-L6-v2) with async thread pool and 3-second timeout to avoid blocking the request.
- Category classification via Ollama JSON mode with keyword fallback.
- Priority suggestion based on urgency keyword detection in complaint text.
- Complaint summarisation via Ollama, used in weekly admin digest emails.

2.3 Dependencies

Dependency	Version	Purpose
Python	3.11+	Primary backend language
Flask	3.x	Web framework — application factory pattern
Flask-SocketIO	Latest	WebSocket real-time push notifications
Flask-CORS	4.x	Cross-Origin Resource Sharing
PyMongo	4.x	MongoDB Python driver
Werkzeug	3.x	Password hashing, file utilities

ReportLab	4.x	PDF generation for complaint receipts
openpyxl	3.x	Excel (.xlsx) export
pyotp	Latest	TOTP two-factor authentication
boto3	Latest	AWS S3 image storage
apscheduler	Latest	Background job scheduling
sentence-transformers	Latest	NLP duplicate detection
python-dotenv	1.x	Environment variable loading
MongoDB	6.x	Primary database
Redis	Latest	WebSocket message broker, session store
Ollama (llama3.2)	Local	Local LLM inference — AI features
Brevo (Sendinblue)	API v3	Transactional email delivery

2.4 Performance Requirements

- Complaint submission (including AI duplicate check) must complete within 5 seconds. The duplicate detection runs in a thread pool with a 3-second timeout; if exceeded, it is skipped gracefully.
- Page load times for all dashboard views must be under 3 seconds under normal load conditions.
- The SLA checker APScheduler job runs every 15 minutes and must complete within its cycle without blocking request handling.
- Ollama AI inference for category suggestion must respond within 10 seconds; the keyword fallback activates if Ollama is unavailable.
- MongoDB queries on complaints are optimised with compound indexes on (user_id, status), (department, status), and (sla_deadline, escalated).
- The system must support concurrent users across citizen, department, and admin portals without data race conditions.

2.5 Hardware Requirements

Component	Minimum	Recommended (Production)
CPU	2 cores	4+ cores (Ollama benefits from more cores)
RAM	4 GB	8 GB (sentence-transformers + Ollama)
Storage	20 GB	100 GB+ (with S3 offload for images)
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 / 24.04 LTS
Network	1 Mbps	100 Mbps (for Brevo email API calls)
GPU	Not required	Optional (Ollama uses CPU by default)

2.6 Constraints and Assumptions

- Single municipality scope: CivicConnect is designed for one municipal body. Multi-tenancy is listed as future work.

- Ollama cold-start latency: llama3.2 takes 2–5 seconds to load on first inference request. No model pre-warming is implemented.
- English-only: All UI text, email templates, and AI responses are in English. Flask-Babel integration for multi-language support is future work.
- Seven predefined departments: the system seeds seven departments at startup. Additional departments can be created by the administrator.
- No mobile app: the REST API and WebSocket infrastructure are complete, but a React Native or Flutter mobile client does not exist yet.
- Brevo API key required: email delivery depends on a valid Brevo account and API key. If Brevo is unreachable, OTP registration fails loudly; other emails fail silently.
- A valid MongoDB instance must be reachable at startup. Redis is optional in development but required in production for multi-worker SocketIO.

Chapter 3: Design

3.1 System Architecture

CivicConnect follows a layered MVC (Model-View-Controller) architecture decomposed across four distinct layers: Client, Application, Data, and Cross-Cutting Security.

Layer	Components	Responsibility
Client Layer	HTML5/CSS3/Vanilla JS, Leaflet.js, Chart.js, Socket.IO Client, 16 Jinja2 Templates	Browser-side rendering, GPS map picking, analytics charts, real-time WebSocket updates
Application Layer	Flask Blueprints: auth.py, complaints.py, admin.py, department.py, ai_proxy.py, notifications.py, sla.py	Route handling, business logic orchestration, session management
Service Layer	email_service.py, ai_service.py, pdf_service.py, export_service.py, storage_service.py, scheduler.py	Stateless business logic — email, PDF, AI, exports, storage, scheduling
Data Layer	MongoDB (civicdb), Redis, AWS S3	Persistent data, session store, WebSocket broker, image storage
Security Layer	csrf.py, rate_limit.py, helpers.py, two_factor.py	CSRF protection, rate limiting, authentication decorators, TOTP 2FA

The Application Factory pattern is used in app.py — the Flask app instance is created and configured in a factory function, making the application easy to test and extend. All seven Blueprints are registered through this factory, along with CSRF hooks, database initialisation, SocketIO setup, and background scheduler startup.

3.2 Use Case Diagram

The Use Case Diagram identifies three primary actors — Citizen, Department Staff, and System Administrator — along with an external AI system actor (Ollama) for AI-assisted features. The diagram maps all major system interactions including complaint submission, status management, analytics, AI assistance, and export functionality.

Key use cases include:

- UC-01: Submit Complaint — Citizen submits a geo-tagged complaint with optional photos; AI checks for duplicates and suggests category; system auto-routes to department and assigns SLA deadline.
- UC-02: Update Complaint Status — Department Staff updates status with comment and evidence photos; citizen notified in real time.
- UC-03: View System Analytics — Administrator views resolution-time metrics, SLA compliance rates, and complaint trends by category and department.
- UC-04: Use AI Assistant — Citizen asks civic questions through the Ollama-powered chatbot.
- UC-05: Export Data — Administrator exports complaint data as CSV or Excel with date-range filters.

- UC-06: Configure SLA Rules — Administrator defines maximum resolution hours per department-category-priority combination.

3.3 Activity Diagram

The Activity Diagram describes the workflow for citizen complaint submission — the primary and most complex use case in the system. The workflow includes the following key steps:

- Citizen logs in and navigates to the complaint submission form.
- Three parallel activities: filling the title and description, pinning the GPS location on the Leaflet map (mandatory), and optionally uploading up to 5 evidence photos.
- On submission, the server validates CSRF token and rate limits, then sanitises all text fields via `html.escape()`.
- AI duplicate detection runs asynchronously (thread pool, 3-second timeout). If similarity > 0.85, a confirmation dialog is shown to the citizen.
- Category-to-department mapping is performed via `CATEGORY_DEPT_MAP`. SLA deadline is computed from `sla_configs`.
- MongoDB insert and audit log entry are written. Brevo email confirmation is dispatched. A 201 JSON response with `complaint_id` and `sla_deadline` is returned to the citizen.

3.4 Sequence Diagram

The Sequence Diagram illustrates the dynamic interaction between all system components during complaint submission — 19 distinct steps from form submission to final confirmation. The five participants are: Citizen, Browser/UI, Flask/Complaints Blueprint, AIService, MongoDB, and EmailService.

Key interaction highlights:

- Steps 1–4: Citizen fills form and submits `POST /api/complaints` with CSRF token. Server validates CSRF, checks rate limits, and sanitises all text fields.
- Steps 5–9: AIService runs cosine similarity (sentence-transformers) for duplicate detection and calls Ollama for category classification — both asynchronously.
- Steps 10–12: `CATEGORY_DEPT_MAP` lookup assigns department. `sla_configs` query computes SLA deadline (default 72 hours if no specific rule exists).
- Steps 13–15: `complaints_col.insert_one()` persists the complaint. `activity_col.insert_one()` creates the audit log entry.
- Steps 16–17: EmailService calls Brevo API to send a confirmation email to the citizen (non-blocking; raises only on OTP failure).
- Steps 18–19: 201 JSON response with `complaint_id` and `sla_deadline` returned to the browser. Citizen sees the confirmation and PDF link.

3.5 Class Diagram

The Class Diagram models the primary classes in CivicConnect, categorised as entity classes (data models), service classes (business logic), and utility classes (cross-cutting concerns).

Class	Type	Key Attributes / Methods
User	Entity	<code>_id</code> , <code>name</code> , <code>email</code> , <code>password</code> (hashed), <code>role</code> , <code>totp_secret</code> , <code>recovery_codes</code> , <code>locked_until</code> <code>register()</code> , <code>login()</code>

Complaint	Entity	_id, complaint_id, user_id, title, description, category, lat, lng, images, priority, status, department, sla_deadline, escalated submit(), updateStatus()
ComplaintActivity	Entity	_id, complaint_id, actor, field, old_value, new_value, comment, timestamp log(), getHistory()
AuthService	Service	rate_limiter, csrf registerCitizen(), verifyOTP(), login(), resetPassword(), verifyTOTP()
ComplaintMgr	Service	ai_service, storage submitComplaint(), autoRoute(), generatePDF(), assignSLADeadline()
AIService	Service	model (SentenceTransformer), ollama_url isDuplicate(), classifyCategory(), summarize(), suggestPriority()
EmailService	Service	brevo_api_key sendOTP(), sendStatusAlert(), sendPasswordReset(), sendSLAAlert(), sendWeeklyDigest()
RateLimiter	Utility	rate_limits_col checkLimit(), incrementCounter(), lockAccount()

3.6 Entity Relationship (ER) Diagram

CivicConnect uses MongoDB as its data store. Although MongoDB is document-oriented and does not enforce relational constraints at the database level, logical entity relationships are modelled below. Referential integrity is enforced at the application layer.

The civicdb database contains seven collections:

Collection	Relationship	Purpose
users	1 → N with complaints (via user_id)	All accounts: citizens, admin, department staff
complaints	1 → N with complaint_activity (via complaint_id)	Core complaint records with SLA and GPS data
complaint_activity	N → 1 with complaints	Immutable audit log of all field changes
otp_verifications	FK → users (email)	Pending OTP records; TTL 10 min
password_resets	FK → users (user_id)	Reset tokens; TTL 30 min
rate_limits	Keyed by (action, ip)	IP-based rate limit counters; TTL at window expiry
sla_configs	Referenced at complaint creation	Admin-defined SLA rules per dept-category-priority

3.7 State Transition Diagram

The State Transition Diagram models the complete lifecycle of a civic complaint. Six states are defined with well-specified transition conditions.

State	Trigger	Description
PENDING	On submission	Initial state. Complaint stored, awaiting department acknowledgement.

IN PROGRESS	Department assigns and starts work	SLA deadline countdown active. Department working on resolution.
RESOLVED	Department marks resolved	Issue fixed. Citizen notified. Resolution time computed for analytics.
REJECTED	Department or admin rejects	Complaint deemed invalid, duplicate, or out of scope.
ESCALATED	APScheduler SLA job (every 15 min)	SLA deadline passed without resolution. Alert emails sent to dept and admin. escalated=true flag set.
DELETED	Citizen deletes own unassigned complaint	Record and associated images removed from storage.

Email notification triggers: PENDING → Brevo confirmation email on submission. IN PROGRESS → status update email with department name. RESOLVED/REJECTED → final status email with resolution comment. ESCALATED → SLA alert email to department head and administrator.

3.8 Database Design

complaints Collection — Key Fields

Field	Type	Description
complaint_id	String	Human-readable ID: CMP-YYYYMMDD-XXXXXX (date + UUID4 hex)
user_id	String	References users._id. Used for ownership checks.
title / description	String	HTML-escaped. Max 120 / 2000 characters.
category	String	Validated against VALID_CATEGORIES whitelist (8 values)
lat / lng	Float	GPS coordinates from Leaflet.js — now mandatory
images	Array	S3 pre-signed URLs or local paths. Max 5 per complaint.
priority	String	Enum: Low Medium High. AI-suggested, user-confirmed.
status	String	Enum: Pending In Progress Resolved Rejected
department	String	Auto-assigned via CATEGORY_DEPT_MAP on submission
sla_deadline	DateTime	Computed from sla_configs. Default: now + 72 hours.
escalated	Boolean	True if SLA deadline passed and status != Resolved

MongoDB Indexes

Collection	Index	Purpose
complaints	{ user_id: 1, status: 1 }	Compound — citizen dashboard queries
complaints	{ department: 1, status: 1 }	Compound — department portal queries
complaints	{ sla_deadline: 1, escalated: 1 }	Compound — SLA checker job

complaint_activity	{ complaint_id: 1, timestamp: -1 }	Activity timeline queries
otp_verifications	{ expires_at: 1 } TTL	Auto-delete after 10 minutes
password_resets	{ expires_at: 1 } TTL	Auto-delete after 30 minutes
rate_limits	{ expires_at: 1 } TTL	Auto-delete at window expiry
rate_limits	{ action: 1, ip: 1 } unique	O(1) rate limit lookups
users	{ email: 1 } unique	Enforces no duplicate accounts

3.9 API Reference

Authentication APIs

Method	Endpoint	Auth	Description
POST	/api/register	None	Submit citizen registration; triggers 6-digit OTP email
POST	/api/verify-otp	None	Verify OTP; creates user account (max 5 attempts)
POST	/api/login	None	Authenticate; redirects to TOTP step if 2FA enabled
POST	/api/2fa/verify	Partial	Verify TOTP code to complete login
POST	/api/2fa/setup	Session	Enable 2FA; returns TOTP QR code
POST	/api/logout	Session	Clear session and invalidate Redis entry
POST	/api/forgot-password	None	Request reset email; always 200 (enumeration prevention)
POST	/api/reset-password	None	Reset password with 32-byte token

Complaint APIs

Method	Endpoint	Auth	Description
POST	/api/complaints	Login	Submit complaint (AI dup check + SLA assignment)
GET	/api/complaints/my	Login	List own complaints with filters/pagination
PATCH	/api/complaints/:id/edit	Login	Edit own unassigned complaint
DELETE	/api/complaints/:id/delete	Login	Delete own unassigned complaint + images
GET	/api/complaints/:id/pdf	Login	Download PDF receipt with QR code
POST	/api/upload-image	Login	Upload evidence image to S3 or local storage

Admin APIs

Method	Endpoint	Auth	Description
GET	/api/admin/complaints	Admin	All complaints with filters incl. date range, overdue
PATCH	/api/admin/complaints/:id	Admin	Update status, priority, department
GET	/api/admin/analytics	Admin	Full analytics incl. resolution-time per department
GET	/api/admin/export/csv	Admin	13-column CSV export with optional date range

GET	/api/admin/export/excel	Admin	Excel export with summary sheet
POST	/api/admin/sla	Admin	Create or update SLA rule
POST	/api/admin/departments/create	Admin	Create new department account

Department APIs

Method	Endpoint	Auth	Description
GET	/api/dept/complaints	Dept	Dept-scoped complaints with SLA urgency indicators
POST	/api/dept/complaints/:id/update	Dept	Update status + comment + evidence photos; emits WebSocket
GET	/api/dept/stats	Dept	Stats incl. SLA compliance rate
GET	/api/dept/sla/overdue	Dept	List escalated complaints past SLA deadline

3.10 Security Architecture

Security Control	Implementation
Password Hashing	Werkzeug pbkdf2:sha256. Passwords are never stored in plaintext.
CSRF Protection	Double Submit Cookie pattern. X-CSRF-Token header matched against csrf_token cookie using secrets.compare_digest() (timing-safe). Token regenerated on every login.
Session Management	Server-side Flask sessions with mandatory SECRET_KEY (64-char hex). Session ID regenerated on login to prevent session fixation. HttpOnly cookies.
IP Rate Limiting	MongoDB TTL collection. Login: 10/15min. Register: 5/hour. OTP: 10/hour. 2FA: 5/15min. Reset: 3/15min.
Account Lockout	10 consecutive login failures per email → 30-minute lockout. Defeats distributed brute-force attacks from multiple IPs.
TOTP Two-Factor Auth	pyotp TOTP (RFC 6238) for admin and department accounts. 30-second window with ±1 drift tolerance. 8 hashed recovery codes per account.
Input Sanitisation	html.escape() on all user text fields before storage. Prevents stored XSS attacks.
Input Validation	Whitelist validation for category, status, priority enums. Length limits on all text fields. GPS coordinates required.
OTP Verification	6-digit OTP, 10-minute TTL, 5-attempt limit. Record deleted on success or lockout.
Password Reset	URL-safe 32-byte cryptographic token. 30-minute TTL. All Redis sessions invalidated on password change.
Security Headers	X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin on all responses.
Image Upload Security	Extension whitelist, size checked by seeking (not Content-Length), UUID-generated filenames to prevent path traversal.
Audit Trail	complaint_activity collection records every field change with actor, old_value, new_value, and UTC timestamp.

CORS Hardening	ALLOWED_ORIGINS driven by environment variable — no hardcoded origins, safe for production deployment.
----------------	--

3.11 Interface Design

The frontend is built as a CSS custom property-driven design system (style.css, 454+ lines), ensuring consistent theming across all 16 HTML templates. Key design tokens include:

Variable	Value	Usage
--accent	#2563EB	Primary interactive colour — buttons, links, highlights
--bg	#F4F6FB	Page background — light, neutral
--surface	#FFFFFF	Card and panel backgrounds
--pending	#F59E0B	Pending status badge — amber
--progress	#3B82F6	In Progress badge — blue
--resolved	#10B981	Resolved badge — green
--danger	#EF4444	Rejected / error state — red
--sla-ok/warn/over	#10B981 / #F59E0B / #EF4444	SLA urgency indicator (green/amber/red)
--ff-head	Playfair Display (serif)	Section headings and display text
--ff-body	DM Sans (sans-serif)	Body text, labels, form fields

Chapter 4: Implementation, Testing, and Maintenance

4.1 Introduction to Technologies Used

CivicConnect is implemented entirely using open-source tools and frameworks, making it cost-effective for municipal deployments. The backend is Python Flask 3.x with PyMongo for database access. The frontend uses plain HTML5, CSS3, and vanilla JavaScript — no heavy frontend framework is required, keeping the application lightweight and fast.

Technology	Version	Role
Python 3.11+	3.11+	Primary backend language — OOP, type hints, async threading
Flask	3.x	Micro web framework — application factory, Blueprint routing
Flask-SocketIO	Latest	WebSocket real-time push — complaint status updates
PyMongo	4.x	MongoDB Python driver — lazy connection, TTL index support
MongoDB	6.x	Primary NoSQL database — 7 collections, 11+ indexes
Redis	Latest	Session store + WebSocket message broker (production)
Ollama (llama3.2)	Local	Local LLM inference — classification, chat, summarisation
sentence-transformers	Latest	NLP model (all-MiniLM-L6-v2) for duplicate detection
ReportLab	4.x	Programmatic A4 PDF generation for complaint receipts
openpyxl	3.x	Excel (.xlsx) generation for admin data exports
pyotp	Latest	TOTP (RFC 6238) two-factor authentication
boto3	Latest	AWS S3 SDK for cloud image storage
apscheduler	Latest	Background jobs — SLA checker (15 min), digest mailer (daily)
Brevo API	v3 REST	Transactional email — OTP, status alerts, digest, resets
Leaflet.js	CDN	Interactive GPS map picker for complaint geolocation
Chart.js	CDN	Analytics charts on admin and department dashboards
Socket.IO Client	CDN	Real-time WebSocket connection in the browser

4.2 Testing Techniques and Test Plans

A pytest-based test suite was added to CivicConnect to validate all critical system functions. Testing follows a unit-testing approach with test isolation using mock objects for MongoDB collections and external services.

Test Module	Coverage Area	Key Test Cases
test_auth.py	Authentication flows	Registration with valid/invalid OTP, login success/failure, 2FA setup and verify, password reset token generation and use, account lockout after 10 failures

test_complaints.py	Complaint CRUD	Submission with valid and missing fields, GPS coordinate validation, category whitelist enforcement, edit and delete of unassigned vs assigned complaints, PDF receipt generation
test_security.py	Security controls	CSRF token mismatch rejection, rate limit enforcement and window reset, XSS sanitisation of stored text fields, session fixation prevention on login
test_ai_service.py	AI service functions	Duplicate detection threshold (< 0.85 = no dup, > 0.85 = dup), category classification with Ollama and keyword fallback, priority suggestion keyword matching

The test suite uses pytest fixtures to create isolated in-memory test databases and mocked Brevo/Ollama responses. Tests are designed to run without a live MongoDB or Ollama instance, making them suitable for CI/CD pipelines.

4.3 Installation Instructions

Development Setup:

- Clone the repository and navigate to the CivicConnect directory.
- Create a virtual environment: `python -m venv venv && source venv/bin/activate`
- Install Python dependencies: `pip install flask flask-cors flask-socketio pymongo werkzeug python-dotenv reportlab openpyxl pyotp boto3 apscheduler sentence-transformers qrcode`
- Install and run Ollama: visit <https://ollama.com>, install, then run: `ollama pull llama3.2 && ollama serve`
- Ensure MongoDB is running locally on port 27017.
- Copy `.env.example` to `.env` and fill in: `SECRET_KEY` (64-char hex), `BREVO_API_KEY`, `BREVO_FROM_EMAIL`, and all `DEPT_PASSWORD_*` variables.
- Start the application: `python app.py` (Flask dev server on port 5000).

Production Hardening:

- Set `FLASK_DEBUG=false`, `HTTPS_ENABLED=true`, `USE_S3=true`.
- Deploy behind nginx or Caddy with TLS termination. Set `TRUST_PROXY=true`.
- Use Gunicorn with eventlet or gevent worker for SocketIO compatibility.
- Run Redis for WebSocket brokering and session invalidation.
- Set `ALLOWED_ORIGINS` to the production domain only.
- Enable 2FA on all admin and department accounts before go-live.

4.4 End User Instructions

For Citizens

- Register at the citizen portal with your name, email, and password. Check your email for the 6-digit OTP and verify within 10 minutes.
- Log in and navigate to 'Submit Complaint'. Fill the form, pin your location on the map (required), optionally upload up to 5 photos, and submit.
- Track your complaint status from 'My Complaints'. You will receive email notifications at every status change.
- Use the AI Chat Assistant on your dashboard to ask civic questions or get help drafting a complaint.
- Download a PDF receipt with QR code for any complaint from the complaint detail view.

For Department Staff

- Log in through the Department portal using the email and password provided by the administrator.
- View complaints assigned to your department. Urgency colour coding (green/amber/red) shows SLA proximity.
- Update complaint status, add a work comment, and optionally upload evidence photos.
- Monitor your SLA compliance rate on the department dashboard.

For Administrators

- Log in through the Admin portal. Enable 2FA from your profile for enhanced security.
- Manage all complaints system-wide. Use the filter panel to find complaints by status, category, department, date range, or overdue flag.
- Configure SLA rules from the SLA Config panel. Set maximum resolution hours per department-category-priority combination.
- Export complaint data as CSV or Excel from the admin panel. Use date-range filters for partial exports.

Chapter 5: Results and Discussions

5.1 Modules Overview

CivicConnect is divided into clearly separated modules, each serving a distinct purpose:

Module	File(s)	Description
Authentication	auth.py	Citizen OTP registration, multi-portal login with TOTP 2FA, forgot/reset password
Complaint Management	complaints.py	Submission with AI validation, auto-routing, SLA assignment, PDF generation, image upload
Admin Panel	admin.py	System-wide complaint management, resolution analytics, SLA config, CSV/Excel export, dept impersonation
Department Portal	department.py	Dept-scoped complaint management with SLA urgency indicators and real-time WebSocket updates
AI Proxy	ai_proxy.py	Ollama inference bridge — chatbot, category classification, duplicate detection, summarisation
Email Service	email_service.py	Brevo API wrapper — OTP, status alerts, password reset, SLA alerts, weekly digest
PDF Service	pdf_service.py	ReportLab A4 styled complaint receipts with QR code
AI Service	ai_service.py	sentence-transformers duplicate detection, Ollama classification and summarisation
Export Service	export_service.py	CSV and Excel complaint export with date-range filtering
Storage Service	storage_service.py	S3/local storage abstraction for images and avatars
Scheduler	scheduler.py	APScheduler — SLA checker every 15 min, daily digest at 08:00
Rate Limiter	rate_limit.py	MongoDB TTL-backed IP rate limiting with account lockout
CSRF Protection	csrf.py	Double Submit Cookie with timing-safe comparison

5.2 User Interface Walkthrough

Citizen Portal

The landing page (index.html) shows a live complaint statistics counter and a clean call-to-action for registration. After login, the citizen dashboard (user_dashboard.html) presents recent complaints with real-time status badges, an AI chat assistant widget, and profile management. The complaint submission form (submit_complaint.html) features the Leaflet GPS map picker (mandatory), an AI-suggested category dropdown, duplicate warning dialog, and photo upload support. The PDF receipt includes the complaint ID, full details, activity timeline, and a scannable QR code.

Department Portal

The department dashboard (dept_dashboard.html) shows a real-time SLA compliance gauge, count of escalated complaints, and average resolution time for the current month. Each complaint card in the list (dept_complaints.html) is colour-coded by SLA urgency — green for on track, amber for approaching, red for overdue. Status updates are applied through an inline update panel with comment and evidence photo upload.

Admin Portal

The admin dashboard (admin_dashboard.html) provides a system-wide view including total complaints, overdue count, SLA compliance rate, and recent activity. The analytics page (admin_analytics.html) includes Chart.js visualisations: a category distribution pie chart, a status doughnut chart, a departmental volume bar chart, and a resolution-time bar chart per department. The export panel supports one-click CSV or Excel download with date-range filtering. The SLA configuration panel allows admins to set or adjust resolution hour rules per department-category-priority combination.

5.3 Backend and Database

The MongoDB civicdb database contains seven collections. Below is a sample of actual complaint documents demonstrating the schema in practice:

Field	Sample Value
complaint_id	CMP-20260415-A3F7B2
title	Pothole on MG Road near Bus Stand
category	Potholes & Roads
department	Roads & Transport
status	In Progress
priority	High
lat / lng	22.7196 / 75.8577
sla_deadline	2026-04-18T10:00:00Z
escalated	false
images	['https://s3.ap-south-1.amazonaws.com/civicbucket/img-uuid1.jpg']
created_at	2026-04-15T10:00:00Z

The complaint_activity collection records every status change with full before/after values, the acting user's email, timestamp, and any work comment or evidence photos — creating a tamper-evident audit trail for every complaint in the system.

Chapter 6: Summary and Conclusions

CivicConnect was built to address a genuine and persistent gap in Indian municipal governance: the absence of a structured, transparent, and accountable civic complaint management system. The platform replaces fragmented, informal complaint processes with a clean digital workflow — from the moment a citizen pins a pothole on a map to the moment a department marks it resolved.

The system delivers on all of its stated objectives. Citizens have a simple, accessible interface with GPS-based reporting, real-time status tracking, AI assistance, and downloadable PDF receipts. Department staff have a focused portal with SLA urgency indicators and evidence upload. Administrators have complete oversight with analytics, bulk exports, SLA configuration, and a weekly digest.

The technical foundation is solid. The Blueprint-based Flask architecture keeps the codebase modular and maintainable. MongoDB with compound indexes handles all query patterns efficiently. The security architecture is notably thorough — CSRF protection, TOTP 2FA, account lockout, session regeneration, XSS sanitisation, and a complete activity audit log together implement genuine defence-in-depth.

The AI integration is practically useful rather than cosmetic. Locally hosted Ollama means citizen data never leaves the municipal server — a meaningful privacy advantage. The sentence-transformers duplicate detection, wired into the submission pipeline with an async timeout, prevents complaint flooding without degrading user experience.

The improvements made during the enhancement phase — activating the AI pipeline, adding SLA tracking, integrating WebSockets, migrating image storage to S3, and hardening security — collectively transformed CivicConnect from a well-structured prototype into a platform genuinely ready for pilot deployment in a real municipality.

Chapter 7: Future Scope

While CivicConnect is production-ready for a single municipality, there are several meaningful directions for future development:

Enhancement	Description
Complaint Upvoting	Allow multiple citizens to upvote the same issue. Auto-escalate complaints reaching 10+ upvotes to surface high-priority community problems.
OpenStreetMap Heatmap	Since GPS coordinates are now consistently captured, a complaint density heatmap can be added using Leaflet.heat or deck.gl for infrastructure planning.
Citizen Satisfaction Rating	Post-resolution rating system for citizens to rate departmental service quality, feeding into department performance scores.
Multi-language Support	Flask-Babel integration with browser Accept-Language auto-detection for Hindi, Marathi, and other regional language UI and email templates.
React Native Mobile App	A mobile client using the existing REST API and WebSocket infrastructure, with offline-first complaint drafting for low-connectivity areas.
Multi-tenancy Architecture	A SaaS architecture serving multiple municipalities on a shared platform with partitioned data — opening the possibility of a subscription-based civic platform.
DigiLocker / Aadhaar Integration	Verified citizen authentication via India's government identity systems, subject to API access approval.
Public Anonymised API	A public endpoint exposing anonymised complaint statistics for third-party civic apps, researchers, and journalists.
SMS Notifications	SMS status alerts via MSG91 (India-focused) or Twilio for citizens in areas with limited internet access but reliable SMS delivery.
Ollama Model Pre-warming	A startup hook that sends a dummy request to Ollama before the server starts accepting user traffic, eliminating the 2–5 second cold-start latency.

References:

- [1] FixMyStreet, mySociety Technical Report, 2012. Available: <https://fixmystreet.com>
- [2] SeeClickFix Platform Documentation, 2023. Available: <https://seeclickfix.com>
- [3] Jan Samasya — Public Grievance Portal, Government of India. Available: <https://pgportal.gov.in>
- [4] Sharma et al., 'Auto Classification of Civic Complaints Using Sentence Transformers', IEEE ICSCCT 2022.
- [5] Gupta & Mehta, 'Local LLMs for Budget-Constrained Municipal AI', Journal of E-Governance Research, vol.11, 2023.
- [6] OWASP Foundation, 'OWASP Top 10 – 2021'. Available: <https://owasp.org/Top10>
- [7] Singh et al., 'Security Vulnerabilities in Indian E-Governance Portals', IJIS, vol.22, 2023.
- [8] Banker et al., MongoDB in Action, 2nd ed., Manning Publications, 2016.
- [9] Chodorow K., MongoDB: The Definitive Guide, 3rd ed., O'Reilly Media, 2019.
- [10] Pallets Projects, Flask Documentation 3.x, 2024. Available: <https://flask.palletsprojects.com>
- [11] Brevo, Transactional Email API v3 Reference, 2024. Available: <https://developers.brevo.com>